

London Metropolitan University



CS7080 Cloud Computing and Internet of Things

Coursework II – Proposal

Groupwork

Smart Shopping Cart using IoT, RFID & Azure Cloud

Group members:

NAME | 20xxxxxx

NAME | 20xxxxxx

Submitted To:

Abstract

The retail industry faces persistent challenges with inefficient checkout processes, long queues, and reactive inventory management. This project proposes a Smart Shopping Cart system leveraging RFID technology, IoT integration, and Microsoft Azure cloud services to deliver seamless self-checkout and real-time cart management. The proposed solution employs a Wokwi-simulated Arduino VE equipped with dual RFID scanners — one for item identification and a second dedicated to contactless payment card authentication. Item scan events, comprising RFID UID, CartId, and Timestamp, are transmitted to Microsoft Azure IoT Hub, which handles device authentication and high-throughput message ingestion. Events are subsequently routed via Azure Event Hub to a Dockerised backend hosted on Azure App Service, enabling decoupled and scalable processing. The backend applies toggle cart logic — adding a product on the first scan and removing it on a repeat scan — before pushing real-time cart updates to an Angular Static Web App via WebSocket. Redis Cache provides low-latency RFID UID lookups, reducing load on the underlying Azure SQL Database, which stores product catalogues, cart sessions, scan events, and transaction records. Azure Key Vault securely manages payment token mappings for mock contactless card processing, while Azure Communication Service delivers OTP-based mobile verification at session initiation and dispatches invoice and confirmation messages post-checkout via Email and SMS. Both actual and synthetic JSON datasets are used to simulate product catalogues during prototyping. This proposal outlines the system architecture, aims, objectives, and scope, demonstrating how cloud-IoT integration can enable an automated, scalable, and cost-effective retail self-checkout experience without cashier intervention.

Problem Statement

Traditional retail checkout processes often involve long waiting lines, manual scanning of items, and a dependency on cashiers. This model is both time-consuming and labour-intensive, and it is inherently prone to human error. During peak shopping hours, queue congestion leads to:

- **Customer frustration and wasted time** — shoppers endure lengthy waits at staffed counters, negatively impacting satisfaction and potentially causing them to abandon purchases altogether.
- **Inefficient store operations** — manual processes require significant staffing investment, and inventory reconciliation through periodic stock-takes is resource-intensive and susceptible to inaccuracies.

- **Limited scalability during peak shopping hours** — traditional checkout systems cannot dynamically scale to accommodate sudden surges in customer volume, resulting in bottlenecks and revenue loss.

There is therefore a clear need for an intelligent, real-time system capable of automatically identifying items as they are placed into a shopping cart, maintaining a live running total, processing contactless payments, managing inventory dynamically, and delivering digital receipts — all without cashier intervention. The Smart Shopping Cart system proposed herein directly addresses these challenges through the integration of RFID sensor technology and Microsoft Azure cloud services.

Aims and Objectives

The main aim of this project is to design and implement a Smart Shopping Cart prototype that automates item detection, real-time billing, contactless payment, and inventory management by integrating RFID-based IoT sensor technology with Microsoft Azure cloud infrastructure and a modern web frontend. The specific objectives to fulfil this aim are:

- Simulate an IoT-enabled shopping cart device using Arduino VE on the Wokwi platform, incorporating two RFID scanners — one for item identification (adding and removing products via toggle logic) and a second dedicated to contactless payment card authentication at checkout.
- Develop an Angular Static Web App as the customer-facing interface, providing session initiation, real-time cart display, checkout confirmation, payment selection, and post-purchase feedback screens.
- Implement OTP-based mobile number verification at session start using Azure Communication Service, enabling authenticated CartSession creation linked to the customer's user record.
- Configure Azure IoT Hub as the cloud gateway to securely ingest RFID scan events (UID, CartId, Timestamp) from the Arduino VE device, and route messages to Azure Event Hub for decoupled, scalable downstream processing with support for multiple consumer groups.
- Develop an Azure App Service backend (containerised via Docker) that consumes events from Event Hub using EventHubConsumerClient, applies cart logic (add on first UID scan, remove on repeated UID scan), and coordinates with the data layer.
- Implement Redis Cache for high-performance RFID UID lookups to avoid repeated SQL database queries on every scan event, improving throughput under concurrent cart usage.
- Establish real-time bidirectional communication between the backend and the Angular frontend using the WebSocket protocol, enabling instant cart updates, quantity changes, and status transitions without page refresh.
- Implement contactless payment processing using a second RFID scanner; map the scanned payment UID to mock card details (masked card number, holder name, expiry, balance) stored securely in Azure Key Vault, and record the completed transaction in the database.
- Automate post-checkout workflows: update transaction and inventory records, generate an itemised invoice, and dispatch the invoice and a greeting message to the customer via

Email and SMS using Azure Communication Service.

- Design a normalised relational database schema encompassing Company, Branch, Cart, User, Product, ProductCategory, ProductRfidItem, CartSession, CartEvent, CartLineItem, and Transaction entities to underpin all system operations.

Purpose

- Enable seamless self-checkout without cashier intervention, reducing operational staffing costs and eliminating checkout queues.
- Provide real-time cart management, allowing customers to add and remove items instantly through RFID scanning with immediate visual feedback on the Angular interface.
- Enable faster payment processing through contactless RFID card scanning, removing the need for manual card entry or staffed payment terminals.
- Improve customer satisfaction by reducing friction at every stage — from authentication through OTP, to guided scanning, to automated receipt delivery.
- Provide enterprise scalability through Azure cloud infrastructure, supporting analytics, real-time inventory tracking, and cloud-based resilience across multiple branches.

Scope of Project

- The hardware component is entirely simulated using the Wokwi virtual platform for Arduino VE; behavioural differences between simulation and physical hardware may affect real-world RFID read performance and deployment fidelity.
- Each physical product unit must be tagged with a unique RFID label whose UID maps to a ProductRfidItem record; the accuracy of item identification depends entirely on correct tag assignment in the database.
- OTP-based mobile verification at session start is dependent on Azure Communication Service and telecommunications infrastructure; network disruptions may prevent users from initiating a cart session.
- WebSocket connectivity between the App Service backend and the Angular frontend requires a stable network; any interruption will break the real-time update stream and may require the user to refresh the session.
- The contactless payment mechanism uses mock card details stored in Azure Key Vault and does not integrate with a real payment gateway or acquiring bank; it is a proof-of-concept simulation of tokenised contactless payment.
- Redis Cache and Azure SQL Database operate as separate managed services; scaling these to support hundreds of simultaneous carts in a large retail environment introduces additional infrastructure management complexity.
- Security and privacy of customer transaction data transmitted across cloud infrastructure requires robust authentication, authorisation, and encryption mechanisms, introducing additional implementation complexity and a potential attack surface.

- Integration with existing retail point-of-sale (POS) systems, enterprise resource planning (ERP) platforms, or customer loyalty programme databases falls outside the current scope and would require additional development and integration effort.
- The current prototype is designed as a single-cart proof-of-concept; the CartSession table design supports multiple concurrent sessions per cart, but full multi-cart concurrency testing is out of scope for this phase.

System Architecture

The Smart Shopping Cart system is structured across six interconnected layers: the User Interface Layer, the Edge Device Layer, the IoT Communication Layer, the Application Layer, the Data Layer, and the Notification Layer. Each layer performs a distinct role in ensuring that shopping cart events are detected, transmitted, processed, stored, and communicated to the end user in real time.

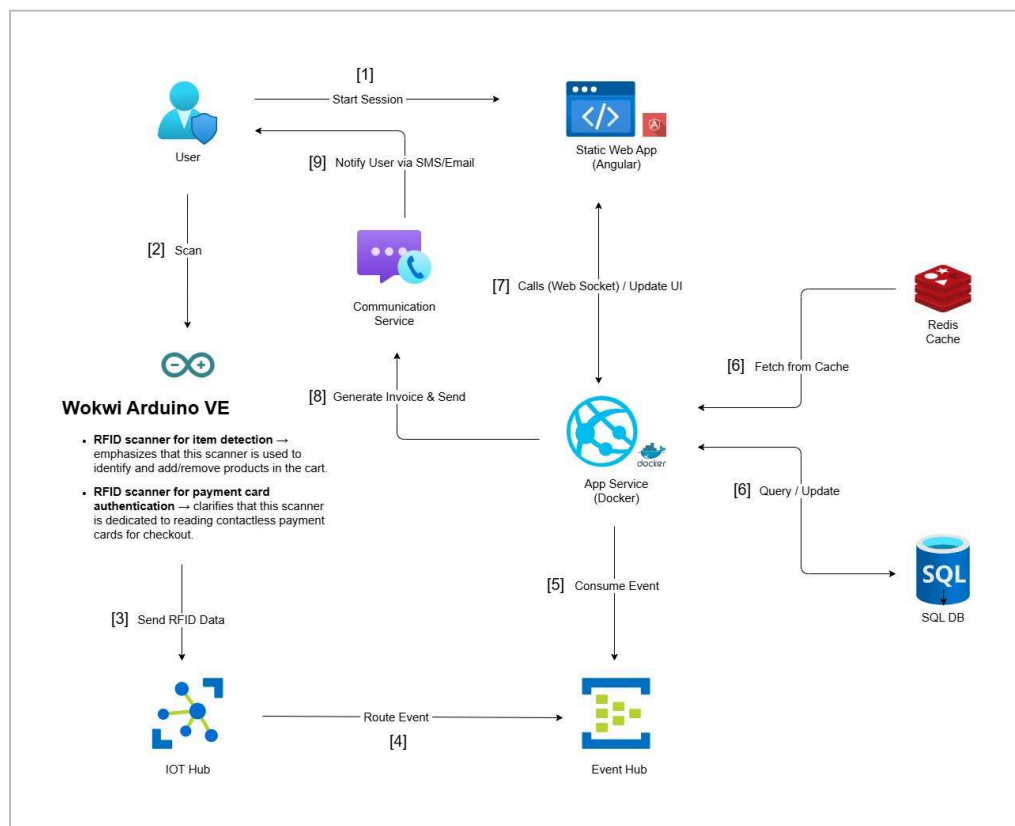


Figure 1: Smart Shopping Cart System Architecture

[1] User Interface Layer — Angular Static Web App: An Angular application deployed as an Azure Static Web App serves as the customer-facing interface. It guides the user through session initiation, mobile OTP verification, real-time cart display, checkout confirmation, payment selection, and post-payment feedback. Updates are received from the backend in real time via WebSocket [7].

[2] Edge Device Layer — Wokwi Arduino VE: An Arduino VE microcontroller, simulated on the Wokwi platform, acts as the embedded IoT device within the shopping cart. It hosts two RFID scanners: the first detects items placed in or removed from the cart by reading product RFID tags; the second reads the customer's contactless payment card at checkout. Wokwi enables rapid prototyping without requiring physical hardware.

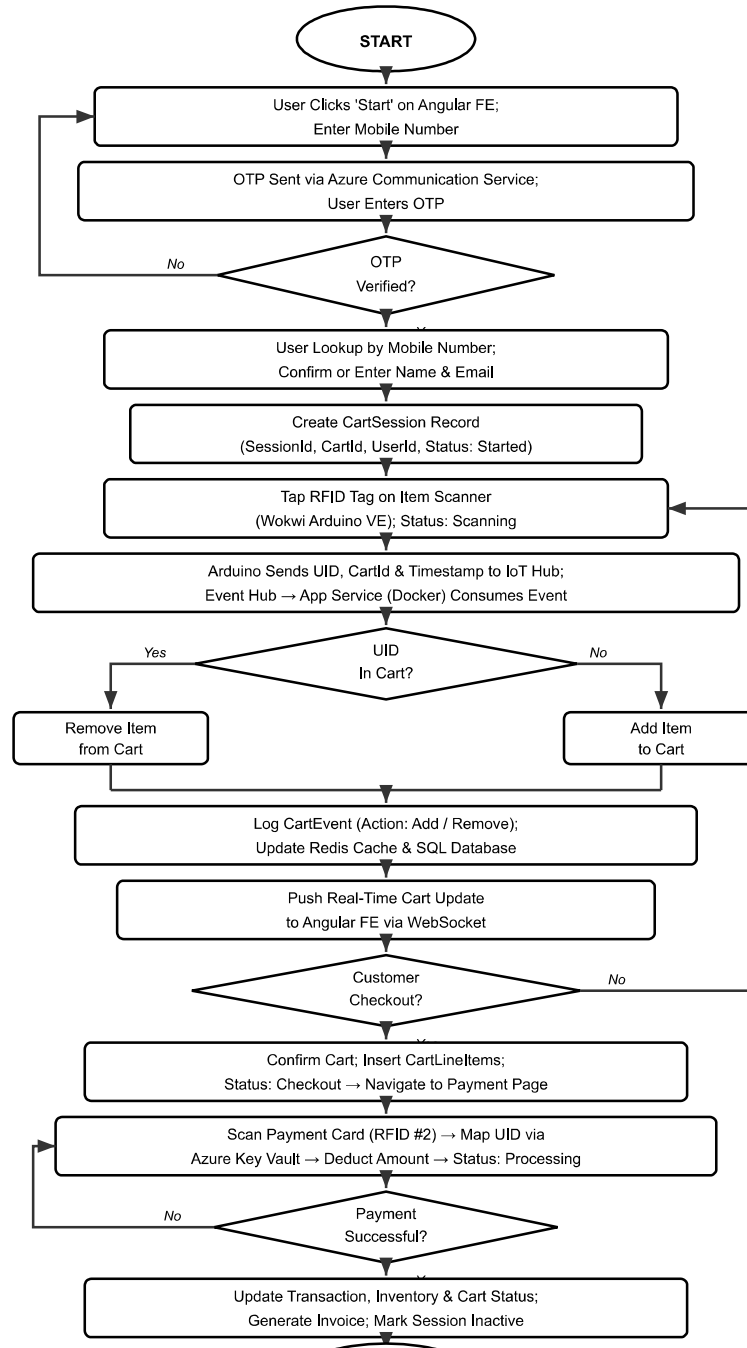
[3–4] IoT Communication Layer — IoT Hub & Event Hub: RFID scan data (UID, CartId, Timestamp) is published from the Arduino VE to Microsoft Azure IoT Hub [3], which provides device authentication and high-throughput message ingestion. IoT Hub routes events to Azure Event Hub [4], introducing a decoupling layer that enables scalability and supports multiple consumer groups for logging and analytics.

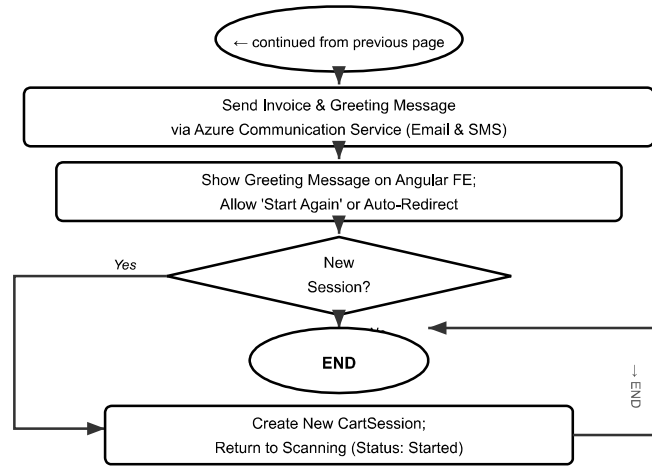
[5–6] Application & Data Layer — App Service, Redis & SQL DB: An Azure App Service running within a Docker container [5] consumes events from Event Hub via EventHubConsumerClient. It resolves RFID UUIDs against Redis Cache [6] for low-latency lookups and falls back to the Azure SQL Database [6] for persistent product and session data. Cart logic (add/remove toggle, CartEvent logging, CartSession status transitions) is handled entirely within this layer. Azure Key Vault manages payment token mappings.

[8–9] Notification Layer — Azure Communication Service: The App Service triggers Azure Communication Service [8] to deliver OTP codes at session initiation and, upon successful payment, to dispatch the itemised invoice and a greeting message to the customer via SMS and Email [9].

Flowchart

Smart Shopping Cart – System Flow



Flowchart (continued)**Smart Shopping Cart – System Flow (Steps 17–End)****Key Design Notes from the System Flow:**

- **Toggle Cart Logic:** Each RFID tag uniquely represents one physical product unit. Scanning a UID for the first time adds the item; scanning the same UID again removes it. This eliminates the need for a separate "remove" button and is tracked via the CartEvent table (Action: Add / Remove).
- **Multi-Session per Cart:** The CartSession table supports multiple sessions per CartId. A new record is inserted for each shopping session; the IsActive flag identifies the current active session. When a scan event arrives at the backend, it resolves the active session for the corresponding CartId.
- **Quantity Handling:** When the same product (same ProductId, different UID) is scanned multiple times, the frontend increments the displayed quantity rather than adding a duplicate line item. The backend sends the product record to the frontend with a quantity-update signal.
- **Redis Cache Strategy:** UID-to-product lookups are first resolved via Redis Cache. A cache miss falls back to the SQL database, and the result is written back to the cache for subsequent requests.

Entity Relationship Diagram

The following ER diagram illustrates the relational database schema underpinning the Smart Shopping Cart system. The schema is designed to support multi-branch, multi-cart deployments with full session traceability, event-level audit logging, and transactional integrity.

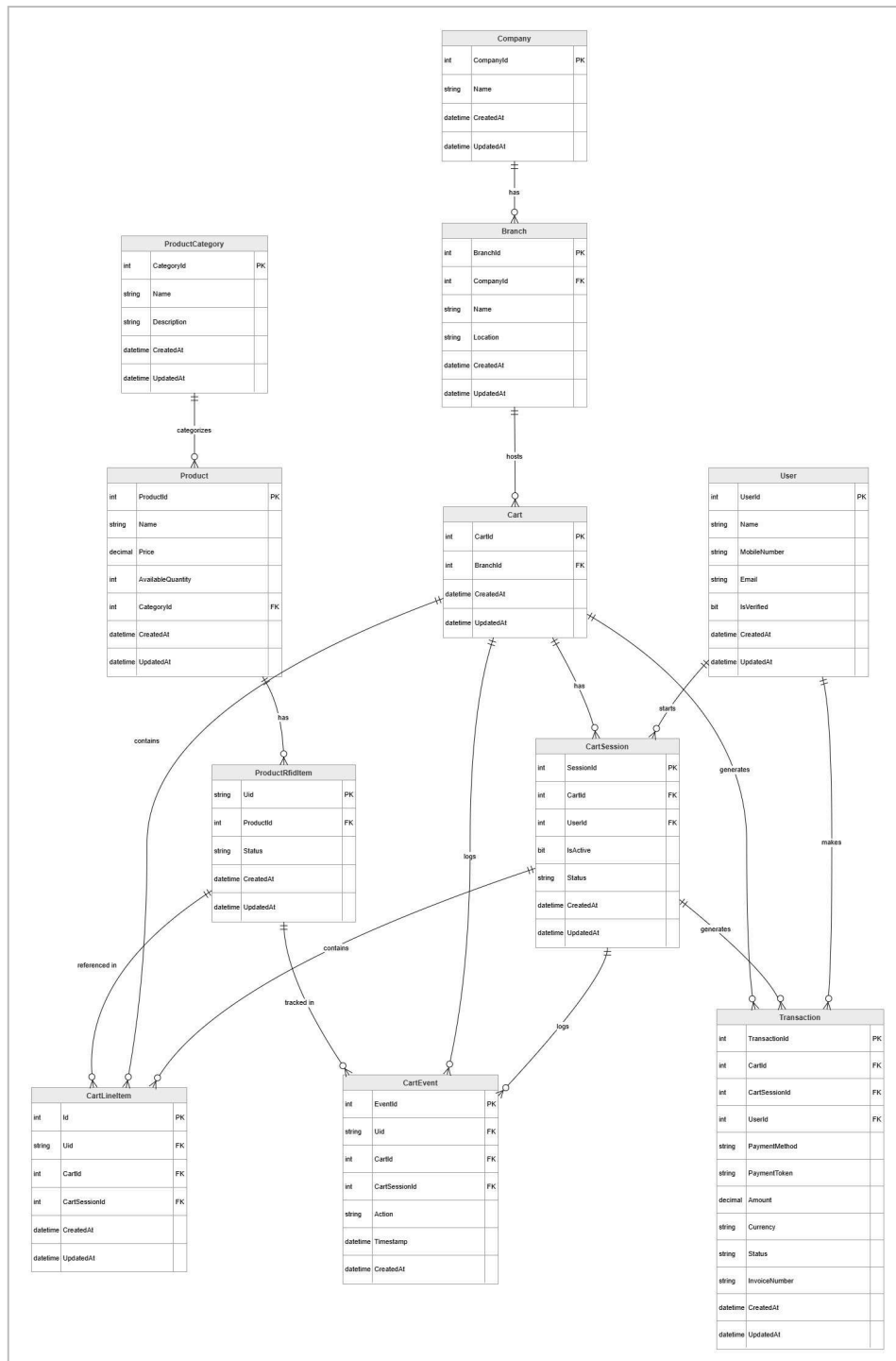


Figure 2: Entity Relationship Diagram — Smart Shopping Cart Database Schema

The schema is anchored at the **Company** and **Branch** level, allowing a single deployment to serve multiple retail locations. Each branch hosts one or more **Cart** devices. Products are organised by **ProductCategory** and each physical product unit carries a unique RFID tag recorded in the **ProductRfidItem** table. A **CartSession** links a **User** to a Cart for the duration of a shopping visit. Every RFID scan event is logged in the **CartEvent** table (Add/Remove). Upon checkout, confirmed items are written to **CartLineItem**, and a completed payment creates a **Transaction** record containing the payment method, token reference, amount, currency, status, and generated invoice number.

Database Design

The following tables constitute the Azure SQL Database schema for the Smart Shopping Cart system. All tables include *CreatedAt* and *UpdatedAt* audit timestamps.

Company

Column	Type	Constraint	Description
CompanyId	int	PK	Unique company identifier
Name	string		Company trading name

Branch

Column	Type	Constraint	Description
BranchId	int	PK	Unique branch identifier
CompanyId	int	FK → Company	Parent company
Name	string		Branch name
Location	string		Physical address

Cart

Column	Type	Constraint	Description
CartId	int	PK	Unique cart identifier (shared by FE and device)
BranchId	int	FK → Branch	Branch that owns this cart

User

Column	Type	Constraint	Description
UserId	int	PK	Unique user identifier
Name	string		Customer full name
MobileNumber	string		Used for OTP verification and SMS
Email	string		Used for invoice delivery
IsVerified	bit		Mobile OTP verification status

ProductCategory / Product / ProductRfidItem

Column	Type	Constraint	Description
CategoryId	int	PK	Product category
ProductId	int	PK	Unique product; holds Name, Price, AvailableQuantity, CategoryId FK
Uid (RFID)	string	PK → ProductRfidItem	RFID tag UID; FK to Product; each unit has a unique tag
Status	string		RFID tag status (Active / Disabled)

CartSession

Column	Type	Constraint	Description
SessionId	int	PK	Unique session per shopping visit
CartId	int	FK → Cart	Cart used in this session
UserId	int	FK → User	Authenticated customer
IsActive	bit		Whether this is the current active session for the cart
Status	string		Started / Scanning / Checkout / PaymentProcessing / Completed

CartEvent

Column	Type	Constraint	Description
EventId	int	PK	Unique scan event log entry
Uid	string	FK → ProductRfidItem	Scanned RFID tag
CartId	int	FK → Cart	
CartSessionId	int	FK → CartSession	
Action	string		Add or Remove
Timestamp	datetime		Event time reported by the device

CartLineItem

Column	Type	Constraint	Description
Id	int	PK	Confirmed cart item at checkout
Uid	string	FK → ProductRfidItem	RFID tag of confirmed item
CartId	int	FK → Cart	
CartSessionId	int	FK → CartSession	

Transaction

Column	Type	Constraint	Description
TransactionId	int	PK	Payment transaction record
CartId	int	FK → Cart	
CartSessionId	int	FK → CartSession	
UserId	int	FK → User	
PaymentMethod	string		Credit/Debit, PayPal, etc.
PaymentToken	string		RFID payment UID (resolved via Azure Key Vault)
Amount	decimal		Total cart value charged
Currency	string		ISO currency code
Status	string		Pending / Completed / Failed
InvoiceNumber	string		Generated invoice reference